Tema File

Preguntas

1. En una aplicación Java se necesita representar la ruta de un directorio, comprobar si existe y listar su contenido, pero sin abrirlo para leer datos. ¿Qué elemento encaja mejor con esa necesidad?

A) `FileReader`, porque permite leer caracteres del directorio.
B) File, porque representa archivos y directorios y permite consultar información sobre ellos.
C) `BufferedReader`, porque mejora el acceso a cualquier recurso del sistema.
D) `ObjectInputStream`, porque sirve para recorrer estructuras del sistema de archivos.

2. Un programa se ejecuta desde `C:/proyecto` y quiere acceder al fichero `pelis.txt` situado en `cine/drama`. Si se desea usar una ruta relativa válida, ¿cuál sería la opción más adecuada?

A) `./C:/proyecto/cine/drama/pelis.txt`
B) cine/drama/pelis.txt
C) `/raiz/cine/drama/pelis.txt`
D) `http://cine/drama/pelis.txt`

3. ¿Qué afirmación describe correctamente una ruta absoluta al trabajar con la clase `File`?

A) Depende siempre del directorio desde el que se ejecuta el programa.
B) Solo puede usarse en sistemas Linux.
C) Indica la ubicación completa del recurso desde la raíz del sistema de archivos.
D) Obliga a utilizar el constructor `File(URI)`.

4. En Java, al trabajar con ficheros de texto, `FileReader` y `BufferedReader` suelen utilizarse conjuntamente porque:

A) ambos son filtros y ninguno accede directamente al fichero.
B) FileReader actúa como iniciador y BufferedReader como filtro que mejora la lectura.
C) `BufferedReader` reemplaza por completo a `FileReader` y no necesita apoyo.
D) ambos escriben y leen a la vez sobre el mismo flujo.

5. Un desarrollador debe guardar varias líneas en `peliculas.txt` y quiere que cada nueva ejecución añada contenido al final sin borrar lo anterior. ¿Qué opción refleja mejor esa necesidad?

A) Crear `new FileWriter(nombreArchivo)` en cada ejecución.

B) Usar new FileWriter(nombreArchivo, true) para abrir el fichero en modo añadido.

C) Sustituir `FileWriter` por `FileReader`.

D) Usar `BufferedReader` y llamar a `readLine()` antes de escribir.

6. En el ejemplo de escritura de películas, ¿qué método se usa para insertar un salto de línea después de cada título?

A) `writeLine()`

B) `append()`

C) newLine()

D) `nextLine()`

7. Si un programa lee un fichero de texto línea a línea hasta llegar al final, ¿qué condición permite detener correctamente el bucle de lectura?

A) Cuando `readLine()` devuelve una cadena vacía.

B) Cuando `readLine()` devuelve `false`.

C) Cuando readLine() devuelve null.

D) Cuando `readLine()` lanza siempre una excepción final.

8. ¿Qué afirmación es correcta sobre los flujos de datos en Java?

A) Un mismo flujo sirve simultáneamente como entrada y salida de forma general.

B) Para leer y escribir un fichero suelen necesitarse dos flujos distintos.

C) Todos los flujos derivan directamente de `File`.

D) Los flujos solo pueden trabajar con archivos, no con otros orígenes o destinos.

9. Dentro de los flujos en formato Unicode de 16 bits, ¿qué pareja de clases abstractas actúa como base del modelo?

A) `InputStream` y `OutputStream`

B) `FileReader` y `FileWriter`

C) Reader y Writer

D) `BufferedReader` y `BufferedWriter`

10. Si una aplicación necesita leer enteros y `double` directamente desde un fichero binario, ¿qué clase resulta más apropiada?

A) `BufferedReader`

B) `ObjectInputStream`

C) DataInputStream
D) `FileWriter`

11. Al serializar un objeto `Alumno` para guardarlo en disco, la clase debe:

A) heredar obligatoriamente de `ObjectOutputStream`.
B) implementar la interfaz Serializable.
C) tener todos sus atributos como `public`.
D) declarar todos sus métodos como `static`.

12. ¿Cuál es la función principal de `serialVersionUID` en una clase serializable?

A) Indicar cuántos objetos pueden guardarse en un fichero.
B) Controlar el tamaño en bytes del objeto serializado.
C) Verificar la compatibilidad entre versiones de la clase durante la deserialización.
D) Convertir automáticamente los atributos `private` en públicos.

13. Un equipo guarda un objeto `Alumno` con una versión de la clase y, semanas después, intenta leerlo con otra versión que añade nuevos atributos. Si las versiones no son compatibles, ¿qué situación describe mejor lo que puede ocurrir?

A) El objeto se recupera siempre sin cambios porque Java ignora las diferencias.
B) Se lanza una InvalidClassException al deserializar.
C) El fichero se convierte automáticamente a texto plano.
D) `FileReader` adapta la estructura del objeto al vuelo.

14. En la serialización de objetos, `ObjectOutputStream` suele trabajar junto a `FileOutputStream` porque:

A) el primero actúa como filtro y el segundo como iniciador del flujo binario.
B) ambos son iniciadores y deben duplicarse por seguridad.
C) `FileOutputStream` solo sirve para texto y `ObjectOutputStream` corrige eso.
D) `ObjectOutputStream` crea directorios automáticamente antes de escribir.

15. ¿Qué sucede con un atributo declarado como `transient` cuando el objeto se deserializa?

A) Conserva siempre el último valor que tenía antes de serializarse.
B) Se elimina también del código fuente de la clase.
C) No se recupera desde el fichero y toma el valor predeterminado de su tipo.
D) Se convierte automáticamente en `static`.

16. En una clase `Alumno`, se decide marcar `notaMedia` como `transient` porque puede recalcularse a partir de otras fuentes. ¿Qué justificación encaja mejor con esa decisión?

A) Evitar que el objeto implemente `Serializable`.
B) Excluir de la serialización un dato no imprescindible para reconstruir el objeto.
C) Forzar que `notaMedia` se escriba dos veces en el fichero.
D) Convertir el fichero resultante en legible como texto.

17. Un alumno crea un directorio con `new File("nuevoDirectorio")` y luego llama a un método para crearlo físicamente. ¿Qué método sería el adecuado según el tema?

A) mkdir()
B) `createDir()`
C) `makeDirectory()`
D) `build()`

18. Si se quiere comprobar que `misArchivos` existe y además es realmente un directorio antes de listar su contenido, ¿qué combinación de comprobaciones es la más apropiada?

A) `canRead()` y `canWrite()`
B) exists() e isDirectory()
C) `length()` y `getName()`
D) `delete()` y `mkdir()`

19. En el entrenamiento sobre productos serializados, tras leer la lista desde `productos.dat`, se muestran solo aquellos con precio mayor que 50. ¿Qué idea didáctica se trabaja con más claridad en ese ejercicio?

A) Que la serialización solo sirve para cadenas de texto.
B) Que, una vez deserializados los objetos, pueden procesarse y filtrarse con lógica de programa.
C) Que `ObjectInputStream` sustituye a las colecciones de Java.
D) Que los ficheros binarios no admiten listas de objetos.

20. ¿Qué ventaja aporta `try-with-resources` en los ejemplos de lectura, escritura y serialización del tema?

A) Impide que se produzca cualquier excepción de entrada/salida.
B) Cierra automáticamente los recursos al finalizar el bloque, incluso si ocurre un error.
C) Convierte cualquier fichero binario en texto legible.
D) Obliga a usar rutas absolutas en lugar de relativas.

Plantilla de respuestas

Nº	Respuesta	Nº	Respuesta
1	B	11	B
2	B	12	C
3	C	13	B
4	B	14	A
5	B	15	C
6	C	16	B
7	C	17	A
8	B	18	B
9	C	19	B
10	C	20	B

Tema XML

Preguntas

1. En una aplicación se necesita un formato de texto que permita estructurar datos jerárquicos, intercambiarlos entre sistemas distintos y definir etiquetas propias. ¿Qué tecnología encaja mejor con esa descripción?

A) HTML, porque está pensada para describir datos personalizados.
B) XML, porque es un lenguaje de marcado extensible orientado a estructurar y transportar datos.
C) CSS, porque organiza información en árbol.
D) SQL, porque define etiquetas para intercambio de documentos.

2. ¿Cuál de las siguientes afirmaciones describe mejor una diferencia esencial entre XML y HTML?

A) XML está pensado principalmente para describir datos, mientras que HTML se centra en mostrarlos.
B) HTML permite etiquetas personalizadas y XML no.
C) XML no puede contener estructura jerárquica.
D) HTML se usa para intercambio de datos y XML solo para diseño visual.

3. Un documento XML válido debe cumplir una condición estructural básica. ¿Cuál es?

A) Incluir siempre una hoja XSLT.

B) Contener exactamente un único elemento raíz.

C) Tener todos los datos guardados como atributos.

D) Empezar siempre con un comentario.

4. En el encabezado `<?xml version="1.0" encoding="UTF-8"?>`, la parte `encoding="UTF-8"` indica principalmente:

A) el nombre del elemento raíz.

B) el número máximo de etiquetas permitido.

C) la codificación de caracteres del documento.

D) que el documento ya está transformado a HTML.

```
5. En un XML como `<persona id="123"><nombre>Ana</nombre></persona>`, ¿qué es
`id="123"`?
```

A) Un nodo de texto.

B) Un atributo del elemento `persona`.

C) Un elemento hijo obligatorio.

D) Una expresión XPath.

6. Un equipo quiere mostrar en el navegador el contenido de un XML con formato de tabla HTML sin modificar el archivo original. ¿Qué opción es la más adecuada?

A) Usar XSLT para transformar el XML a HTML.

B) Sustituir XML por un comentario.

C) Convertir el XML en atributo.

D) Aplicar solo DOM sin ninguna transformación.

7. ¿Qué característica describe correctamente a XSLT?

A) Es un lenguaje imperativo para compilar clases Java.

B) Es un lenguaje declarativo basado en plantillas para transformar documentos XML.

C) Solo permite generar otros documentos XML.

D) No puede usar expresiones XPath.

8. En una hoja XSLT, ¿qué elemento se utiliza para recorrer varios nodos seleccionados, por ejemplo una lista de libros o estudiantes?

A) `xsl:choose`

B) `xsl:template`

C) xsl:for-each

D) `xsl:output`

9. Si en una transformación XSLT se desea generar HTML como resultado, ¿qué instrucción está más directamente relacionada con definir ese formato de salida?

A) `xsl:value-of`

B) xsl:output method="html"

C) `xsl:if test="html"`

D) `xsl:call-template name="html"`

10. XPath se utiliza principalmente para:

A) compilar documentos XML en bytecode.

B) navegar y seleccionar nodos dentro de un documento XML.

C) validar automáticamente cualquier XML sin reglas previas.

D) cifrar el contenido textual de un nodo.

11. Si se quiere seleccionar el título del primer libro de una biblioteca, ¿qué expresión encaja con el ejemplo trabajado en el tema?

A) /biblioteca/libro[1]/titulo

B) `titulo/libro[1]/biblioteca`

C) `//titulo[libro=1]`

D) `biblioteca.titulo.primero`

12. Un profesor necesita obtener solo los nombres de estudiantes con edad mayor que 21 a partir de un XML. ¿Qué idea expresa mejor el uso de XPath en este caso?

A) Permite filtrar nodos mediante predicados según condiciones.

B) Solo sirve para contar elementos, no para seleccionarlos.

C) Solo funciona dentro de HTML, no en XML.

D) Obliga a recorrer todos los nodos manualmente con Java.

13. ¿Qué función XPath permite contar cuántos nodos hay en un conjunto seleccionado?

A) `size()`

B) `length()`

C) count()

D) `total()`

14. En el parser DOM, ¿qué representa normalmente la interfaz `Document`?

A) Un atributo individual de un nodo.
B) El documento XML completo cargado en memoria.
C) Una hoja XSLT pendiente de compilar.
D) El conjunto de excepciones del analizador.

15. ¿Qué ventaja ofrece DOM frente a otros enfoques cuando se trabaja con un XML pequeño o mediano que debe modificarse en varias partes?

A) No carga el documento en memoria.
B) Solo permite acceso secuencial.
C) Facilita el acceso y la modificación de cualquier parte del árbol en cualquier momento.
D) Funciona únicamente con documentos sin atributos.

16. ¿Cuál es la principal desventaja del parser DOM al trabajar con archivos XML muy grandes?

A) No soporta documentos con elemento raíz.
B) Exige usar XSLT en todas las operaciones.
C) Consume más memoria al cargar el documento completo.
D) No permite leer texto contenido en nodos.

17. En Java, si se quiere crear el analizador que parseará un XML con DOM, ¿qué secuencia de clases es la más adecuada según el tema?

A) `TransformerFactory` y `Transformer`
B) DocumentBuilderFactory y DocumentBuilder
C) `NodeList` y `Element`
D) `XPathFactory` y `XPathResult`

18. Después de parsear un XML, un programa quiere obtener el nodo raíz para empezar a recorrer el árbol. ¿Qué método se usa sobre `Document`?

A) `getRootNode()`
B) `getFirstChildElement()`
C) getDocumentElement()
D) `getMainTag()`

19. En un ejercicio Java con DOM, se obtiene `NodeList listaEstudiantes = doc.getElementsByTagName("estudiante")`. ¿Qué utilidad práctica tiene esa llamada?

A) Borra todos los nodos `estudiante` del documento.
B) Recupera una colección de elementos con esa etiqueta para poder iterarlos.

C) Transforma el XML a HTML automáticamente.
D) Convierte cada estudiante en atributo del nodo raíz.

20. Si un programa crea un XML nuevo en memoria y quiere guardarlo físicamente como `agenda.xml`, ¿qué tecnología del tema interviene de forma directa en ese paso final?

A) Transformer, para convertir el DOM en archivo físico.
B) `BufferedReader`, para escribir el árbol DOM.
C) `FileWriter`, porque es la única clase válida para XML.
D) `NodeList`, para exportar el documento a disco.

Respuestas

Nº	Respuesta	Nº	Respuesta
1	B	11	A
2	A	12	A
3	B	13	C
4	C	14	B
5	B	15	C
6	A	16	C
7	B	17	B
8	C	18	C
9	B	19	B
10	B	20	A

Tema JDBC – DDL y DDM

Preguntas

1. En una aplicación Java orientada a objetos que necesita guardar información en una base de datos relacional, aparece el llamado "desfase objeto-relacional". ¿Qué idea resume mejor ese problema?

A) Que Java no puede conectarse a bases de datos relacionales.
B) Que los modelos de objetos y tablas no encajan de forma directa y hay que traducir entre ambos mundos.

C) Que SQL solo funciona con lenguajes funcionales.
D) Que las bases de datos relacionales no permiten relaciones entre datos.

2. En un ejemplo del tema, una clase `Equipo` contiene una colección de `Jugador` y se pasa a un diseño relacional. ¿Qué estructura refleja mejor esa traducción a base de datos?

A) Una única tabla con todos los campos de equipo y jugadores mezclados.
B) Dos tablas, Equipo y Jugador, relacionadas mediante una clave foránea desde Jugador hacia Equipo.
C) Una tabla por cada atributo, todas sin relaciones.
D) Un único fichero de texto sin claves.

3. En la definición de una clase `Persona` con un atributo `Direccion` como objeto anidado, ¿qué reto típico se destaca al llevar esto a una tabla relacional?

A) Que los SGBD generan automáticamente el tipo `Direccion`.
B) Que hay que descomponer el objeto Direccion en varias columnas simples como calle, número, ciudad y código postal.
C) Que solo se puede guardar el nombre de la clase `Direccion` en una columna.
D) Que SQL no admite columnas de tipo texto.

4. JDBC se describe en el tema como:

A) Un sistema gestor de bases de datos propio de Java.
B) Una API que define cómo las aplicaciones Java acceden a bases de datos relacionales mediante drivers específicos.
C) Un lenguaje alternativo a SQL.
D) Un protocolo de red genérico sin relación con bases de datos.

5. ¿Qué papel desempeña un driver JDBC de tipo 4 en la arquitectura de acceso a datos?

A) Actúa como puente JDBC-ODBC usando librerías nativas externas.
B) Es código Java puro que traduce las llamadas JDBC directamente al protocolo nativo del SGBD.
C) Solo sirve para bases de datos en memoria.
D) Necesita siempre un middleware intermedio para funcionar.

6. ¿Cuál de las siguientes interfaces de JDBC se utiliza para representar una conexión activa con la base de datos y gestionar transacciones?

A) `Driver`
B) Connection

C) `Statement`

D) `ResultSet`

7. En los ejemplos del tema se crean tablas con JDBC usando `Statement`. ¿Qué método se emplea para ejecutar sentencias DDL como `CREATE TABLE` o `ALTER TABLE`?

A) `executeQuery()`

B) executeUpdate()

C) `executeDDL()`

D) `executeResult()`

8. En el código de conexión a MySQL con JDBC, la URL tiene el formato `jdbc:mysql://localhost:3306/Ferreteria?serverTimezone=UTC`. ¿Por qué se incluye el parámetro `serverTimezone=UTC`?

A) Para activar el modo lectura-solo.

B) Para evitar problemas relacionados con la zona horaria entre Java y MySQL.

C) Para forzar que la base de datos se cree si no existe.

D) Para habilitar el uso de transacciones.

9. En el ejemplo de creación de la tabla `Productos`, la columna `idProducto` se declara como `INT AUTOINCREMENT PRIMARY KEY`. ¿Qué efecto tiene esta combinación en la práctica?

A) Permite valores nulos y duplicados.

B) Genera automáticamente un identificador único para cada fila nueva y lo usa como clave primaria.

C) Impide insertar cualquier registro sin especificar manualmente el id.

D) Solo sirve para tablas temporales.

10. Una empresa quiere añadir una columna `fechaRegistro` a la tabla `Clientes` existente desde su código Java. ¿Qué tipo de sentencia SQL refleja mejor esa operación de cambio de estructura?

A) `INSERT INTO Clientes (fechaRegistro) VALUES (CURRENT_DATE)`

B) `UPDATE Clientes SET fechaRegistro = CURRENT_DATE`

C) ALTER TABLE Clientes ADD COLUMN fechaRegistro DATE

D) `CREATE INDEX fechaRegistro ON Clientes(fechaRegistro)`

11. Los índices se comparan en el tema con el índice de un libro. En la base de datos, su objetivo principal es:

A) reducir el tamaño físico de la tabla.

B) mejorar la velocidad de búsqueda y filtrado sobre las columnas indexadas.

C) impedir que se inserten filas duplicadas.
D) eliminar automáticamente los registros antiguos.

12. ¿Qué efecto práctico puede tener un uso excesivo de índices sobre una tabla con muchas operaciones de inserción y actualización?

A) Acelera tanto lecturas como escrituras sin coste.
B) Puede ralentizar las operaciones de escritura porque hay que mantener los índices actualizados.
C) Impide la ejecución de transacciones.
D) Desactiva las claves foráneas.

13. Al definir una tabla `Pedidos` con una columna `idCliente` que referencia a `Clientes(idCliente)`, ¿qué tipo de elemento de diseño se está aplicando?

A) Un índice compuesto.
B) Una restricción FOREIGN KEY para asegurar integridad referencial.
C) Una restricción `CHECK` sobre el total del pedido.
D) Un valor por defecto en la columna.

14. En el ejemplo de restricciones sobre la tabla `Pedidos`, se define `total DECIMAL(10, 2) NOT NULL CHECK (total > 0)`. ¿Qué objetivo tiene la parte `CHECK (total > 0)`?

A) Permitir totales negativos.
B) Asegurar que solo se aceptan totales mayores que cero.
C) Generar el total automáticamente.
D) Convertir el total a entero.

15. Un programa Java obtiene una conexión y luego crea un `Statement` para ejecutar `DROP TABLE IF EXISTS Proyectos`. ¿Qué ventaja aporta el uso de `IF EXISTS` en este contexto?

A) Hace que la tabla se recree automáticamente.
B) Evita errores si la tabla ya ha sido eliminada previamente.
C) Convierte la tabla en temporal.
D) Asegura que se eliminen también todas las bases de datos.

16. ¿Qué interfaz de JDBC se utiliza para representar el resultado de una consulta `SELECT`, permitiendo recorrer fila a fila y obtener los valores de las columnas?

A) `Statement`
B) `PreparedStatement`
C) ResultSet
D) `DatabaseMetaData`

17. En una aplicación que ejecuta muchas veces la misma sentencia SQL con parámetros distintos, ¿qué interfaz de JDBC es más adecuada para mejorar rendimiento y seguridad?

A) `Statement`
B) PreparedStatement
C) `CallableStatement`
D) `ResultSetMetaData`

18. ¿Qué combinación de elementos refleja mejor una buena práctica al trabajar con JDBC y DDL según el tema?

A) Crear la conexión, ejecutar el DDL y dejar los recursos abiertos para reutilizarlos siempre.
B) Usar try-with-resources para cerrar automáticamente Connection y Statement después de ejecutar el DDL.
C) Ejecutar todo el DDL dentro de un `ResultSet`.
D) Evitar el manejo de excepciones para no ralentizar la aplicación.

19. En el entrenamiento sobre la tabla `Empleados`, se añade la columna `salario DECIMAL(10,2) DEFAULT 1000`. ¿Qué aporta el uso de `DEFAULT 1000` en este caso?

A) Obliga a que todos los empleados tengan siempre salario 1000.
B) Asigna 1000 solo cuando ya exista un salario previo.
C) Asigna 1000 automáticamente si no se proporciona salario al insertar un empleado.
D) Convierte el salario a entero al guardarlo.

20. Una vez creada correctamente la tabla `Proyectos` con `idEmpleado` como clave foránea hacia `Empleados(idEmpleado)`, ¿qué comportamiento se espera si se intenta insertar un proyecto con un `idEmpleado` que no existe?

A) Se inserta el proyecto igualmente y luego se crea el empleado.
B) La sentencia INSERT falla por violar la restricción de clave foránea.
C) El SGBD sustituye el `idEmpleado` por cero.
D) El proyecto se guarda, pero sin relación con ningún empleado.

Respuestas

Nº	Respuesta	Nº	Respuesta
1	B	11	B
2	B	12	B

3	B	13	B
4	B	14	B
5	B	15	B
6	B	16	C
7	B	17	B
8	B	18	B
9	B	19	C
10	C	20	B

Tema 5
Tema Hibernate

Preguntas

1. En una aplicación Java orientada a objetos que necesita persistir datos en tablas relacionales, ¿qué problema intenta resolver principalmente el mapeo objeto-relacional?

A) Que Java no pueda ejecutar código SQL.
B) Que exista una diferencia entre cómo se modelan los datos en objetos y cómo se almacenan en tablas.
C) Que las bases de datos relacionales no permitan claves primarias.
D) Que Hibernate sustituya al compilador de Java.

2. Cuando una clase `Cliente` en Java tiene atributos como `id`, `nombre` y `email`, y se desea guardar en base de datos con Hibernate, ¿qué representa normalmente el proceso de mapeo?

A) Convertir la clase en una interfaz funcional.
B) Relacionar la clase con una tabla y cada atributo con una columna.
C) Convertir cada atributo en una base de datos distinta.
D) Eliminar la necesidad de clave primaria.

3. ¿Cuál es la ventaja más clara de utilizar un framework ORM como Hibernate frente al acceso directo con JDBC puro?

A) Elimina por completo la necesidad de una base de datos.
B) Permite trabajar con objetos Java y reduce parte del código repetitivo de persistencia.
C) Impide hacer operaciones `SELECT`.
D) Obliga a usar únicamente bases de datos NoSQL.

4. Un framework ORM se define mejor como:

A) Una herramienta que transforma clases Java en interfaces gráficas.
B) Un conjunto de librerías que facilita la correspondencia entre objetos y tablas relacionales.
C) Un servidor de aplicaciones para desplegar proyectos web.
D) Un editor visual de bases de datos sin código.

5. En Hibernate, una clase de entidad es normalmente una clase Java que:

A) solo contiene métodos estáticos.
B) representa información persistente del dominio y se asocia a una tabla.
C) no puede tener atributos privados.
D) se usa exclusivamente para conexiones de red.

6. ¿Qué anotación se utiliza habitualmente para indicar que una clase Java es una entidad persistente en Hibernate?

A) `@Column`
B) @Entity
C) `@TableField`
D) `@PersistentClass`

7. Si una entidad `Producto` necesita marcar su identificador principal, ¿qué combinación de anotaciones es la más habitual?

A) `@Entity` y `@JoinColumn`
B) @Id y, en muchos casos, @GeneratedValue
C) `@Basic` y `@Transient`
D) `@Version` y `@Override`

8. En una entidad Hibernate, la anotación `@Column(name = "precio_unitario")` se usa principalmente para:

A) crear una segunda clase de apoyo.
B) indicar el nombre exacto de la columna con la que se mapea el atributo.
C) abrir la conexión con la base de datos.
D) ejecutar automáticamente un `INSERT`.

9. Un desarrollador define una clase de entidad con varios atributos, pero uno de ellos se calcula en tiempo de ejecución y no debe persistirse. ¿Qué solución es la más adecuada?

A) Marcarlo con @Transient.
B) Marcarlo con `@Id`.
C) Convertirlo en `static` obligatoriamente.
D) Guardarlo siempre como clave ajena.

10. En la configuración de Hibernate, ¿qué componente se utiliza normalmente para crear sesiones de trabajo contra la base de datos?

A) SessionFactory
B) `ResultSetFactory`
C) `EntityCursor`
D) `DriverManagerHibernate`

11. ¿Qué idea describe mejor una `Session` en Hibernate?

A) Un fichero XML donde se guardan las consultas.
B) Un objeto que representa una conversación de trabajo con la base de datos para gestionar entidades.
C) Un tipo de tabla temporal.
D) Un sustituto del modelo entidad-relación.

12. En un proyecto Java con Hibernate, la configuración de conexión suele incluir datos como URL, usuario, contraseña y dialecto SQL porque:

A) Hibernate necesita conocer cómo conectarse al SGBD y cómo generar SQL compatible.
B) esos datos solo sirven para dibujar diagramas UML.
C) sin ellos Hibernate funciona únicamente en memoria.
D) el dialecto reemplaza a las entidades Java.

13. En una operación de alta de un nuevo objeto `Empleado`, ¿qué acción corresponde conceptualmente al CREATE del CRUD?

A) Cargar una entidad existente para mostrarla.
B) Persistir una nueva entidad en la base de datos.
C) Borrar una fila por su identificador.
D) Cerrar la `SessionFactory`.

14. Si una aplicación recupera un objeto `Pedido` desde la base de datos a partir de su identificador, ¿qué operación CRUD está realizando?

A) Create
B) Update
C) Read
D) Delete

15. Un usuario modifica el correo electrónico de un `Cliente` ya existente y se sincroniza el cambio con la base de datos. ¿Qué operación CRUD representa mejor esta situación?

A) Create
B) Read
C) Update
D) Delete

16. ¿Cuál sería el equivalente al DELETE del CRUD en Hibernate?

A) Insertar una nueva entidad con clave nula.
B) Eliminar una entidad persistente de la base de datos.
C) Consultar todos los registros de una tabla.
D) Añadir una anotación a la clase.

17. En una aplicación que usa Hibernate, ¿por qué suele recomendarse abrir una transacción antes de operaciones de escritura como guardar, actualizar o eliminar?

A) Porque sin transacción Java no puede compilar.
B) Porque permite asegurar consistencia y confirmar o deshacer cambios si ocurre un error.
C) Porque reemplaza el uso de entidades.
D) Porque evita definir la clave primaria.

18. Si un desarrollador olvida cerrar la `Session` al terminar una operación, ¿qué problema práctico puede aparecer?

A) Las anotaciones dejan de funcionar en tiempo de compilación.
B) Puede haber fugas de recursos y un uso ineficiente de conexiones.
C) Todas las tablas se eliminan automáticamente.
D) Hibernate convierte las entidades en clases abstractas.

19. En una relación entre `Departamento` y `Empleado`, donde un departamento tiene muchos empleados, ¿qué utilidad aporta Hibernate frente a una gestión manual completa con SQL?

A) Facilita mapear relaciones entre objetos y tablas con menos código repetitivo.
B) Impide trabajar con claves foráneas.

C) Obliga a guardar todos los datos en una sola tabla.
D) Hace innecesario el diseño del modelo de datos.

20. ¿Qué opción resume mejor una buena práctica general al trabajar con Hibernate en operaciones CRUD?

A) Abrir sesiones sin cerrarlas para reutilizarlas indefinidamente.
B) Usar entidades bien anotadas, sesiones controladas y transacciones para las operaciones que modifican datos.
C) Evitar las claves primarias porque Hibernate las genera siempre sin configuración.
D) Reemplazar todas las consultas por cadenas escritas directamente en consola.

Plantilla de respuestas

Nº	Respuesta	Nº	Respuesta
1	B	11	B
2	B	12	A
3	B	13	B
4	B	14	C
5	B	15	C
6	B	16	B
7	B	17	B
8	B	18	B
9	A	19	A
10	A	20	B

Hibernate relaciones 7

1. En el contexto de Hibernate, ¿qué describe mejor una relación «uno a uno» entre dos entidades, por ejemplo `Persona` y `DetallePersona`?

A) Una persona puede tener muchos detalles y cada detalle pertenece a muchas personas.

B) Cada persona se asocia, como máximo, con un único detalle y viceversa.

C) No existe relación entre las tablas, solo entre las clases.

D) Una persona se relaciona con todos los detalles de la base de datos.

2. 2. ¿Qué anotación se utiliza habitualmente en Hibernate para declarar una relación uno a uno en una entidad?

A) `@OneToMany`

B) `@ManyToMany`

C) `@OneToOne`

D) `@EmbeddedId`

3. 3. En una relación uno a uno, cuando se define en la entidad `Persona` un atributo `detalle` de tipo `DetallePersona`, ¿qué anotación se usa normalmente para indicar la columna que actúa como clave foránea?

A) `@JoinColumn`

B) `@Column`

C) `@IdClass`

D) `@SecondaryTable`

4. 4. En una relación «uno a muchos» entre `Cliente` y `Pedido`, ¿qué patrón de mapeo es más común en Hibernate?

A) Una lista de clientes dentro de la entidad `Pedido`.

B) Una colección de pedidos en `Cliente` anotada con `@OneToMany` y, en `Pedido`, un atributo `cliente` anotado con `@ManyToOne`.

C) Dos entidades sin ninguna anotación de relación.

D) Un único campo entero en `Cliente` con el número de pedidos.

5. 5. En una asociación uno a muchos, ¿qué lado suele ser el propietario de la relación en Hibernate para efectos de la clave foránea en la base de datos?

A) La entidad que tiene la colección.

B) Siempre la entidad con más columnas.

C) Normalmente el lado `@ManyToOne`, es decir, la entidad que contiene la clave foránea.

D) Ninguno, Hibernate decide aleatoriamente.

6. 6. ¿Qué anotación es típica para mapear una relación muchos a muchos entre entidades `Alumno` y `Curso` en Hibernate?

A) `@ManyToOne`

B) `@OneToOne`

C) `@ManyToMany`

D) `@ElementCollection`

7. En una relación muchos a muchos entre `Alumno` y `Curso`, ¿qué papel desempeña normalmente la tabla intermedia en la base de datos?

A) Guardar solo estadísticas sin claves.

B) Almacenar pares de claves (`idAlumno`, `idCurso`) que representan cada matrícula.

C) Sustituir a las tablas `Alumno` y `Curso`.

D) Actuar como copia de seguridad de la base de datos.

8. Al definir una relación muchos a muchos en Hibernate, ¿qué anotación se usa habitualmente para indicar el nombre de la tabla intermedia y las columnas que enlazan ambas entidades?

A) `@JoinColumn`

B) `@JoinTable`

C) `@SecondaryTable`

D) `@TableGenerator`

9. En una consulta sobre una relación uno a uno, si se quiere obtener la `Persona` junto con su `DetallePersona` usando Hibernate, ¿qué enfoque es habitual?

A) Ejecutar dos consultas SQL manuales sin usar asociaciones.

B) Acceder a la propiedad `persona.getDetalle()` aprovechando la relación mapeada.

C) Guardar los detalles en un `Map` estático global.

D) Solo es posible con procedimientos almacenados.

10. En el caso de una relación uno a muchos, una consulta típica podría recuperar un `Cliente` y luego acceder a sus pedidos. ¿Cómo se suele hacer esto desde código usando Hibernate?

A) Cargando el cliente y recorriendo la colección `cliente.getPedidos()` si está mapeada con `@OneToMany`.

B) Consultando directamente la tabla de unión sin entidades.

C) Modificando manualmente las claves foráneas en cada pedido.

D) Clonando el objeto cliente para cada pedido.

11. Al trabajar con una relación muchos a muchos, ¿qué beneficio principal aporta tener la asociación bien mapeada en las entidades frente a gestionar solo la tabla intermedia desde SQL manual?

A) Evita completamente el uso de transacciones.

B) Impide que se puedan borrar filas de la tabla intermedia.

D) Obliga a no usar claves primarias.

D) Permite trabajar con colecciones de objetos (`alumno.getCursos()`) en lugar de construir y mantener explícitamente todas las combinaciones de claves.

12. En el contexto de Hibernate, una primary key compuesta se utiliza cuando:

A) Se quiere que la clave primaria tenga un valor de tipo `String`.

B) Una fila se identifica de forma única por la combinación de varios campos.

C) No existe ninguna clave primaria.

D) Se desea que la tabla no tenga índices.

13. ¿Qué enfoque ofrece Hibernate para mapear claves primarias compuestas en una entidad?

A) Utilizar `@EmbeddedId` o `@IdClass` para representar la combinación de campos clave.

B) Usar `@GeneratedValue` sin más.

C) Declarar todos los campos como `@Transient`.

D) Crear siempre una tabla intermedia adicional.

14. 14. En una relación muchos a muchos con información adicional en la tabla intermedia (por ejemplo, fecha de matrícula), ¿qué diseño suele recomendarse?

A) Mantener la relación muchos a muchos y añadir columnas extra sin cambiar nada más.

B) Convertir la tabla intermedia en una entidad propia (por ejemplo, `Matricula`) con sus propias relaciones `@ManyToOne` a `Alumno` y `Curso`.

C) Eliminar las claves foráneas para flexibilizar la estructura.

D) Guardar la información adicional en un fichero externo.

15. Cuando se mapea una relación uno a muchos con `List` o `Set` en Hibernate, ¿qué detalle práctico conviene tener en cuenta a la hora de cargar datos?

A) Que las colecciones no pueden estar vacías.

B) Que puede configurarse la estrategia de carga (perezosa o ansiosa) y esto afectará a cuántas consultas se generan.

C) Que siempre se cargan todos los datos en memoria de golpe, sin configuraciones posibles.

D) Que la colección debe ser siempre de tipo `Vector`.

16. En una relación uno a uno mapeada con Hibernate, ¿qué suele significar marcar el atributo con `optional = false`?

A) Que la columna asociada puede ser nula.

B) Que Hibernate ignorará esa relación.

C) Que la asociación es obligatoria y no se permiten filas sin la relación correspondiente.

D) Que la relación se convierte en muchos a muchos.

17. En una consulta sobre relaciones uno a muchos, ¿qué problema puede aparecer si no se planifica bien la estrategia de carga (`fetch`)?

A) Que la base de datos se vuelva de solo lectura.

B) El conocido problema de N+1 consultas, donde se lanza una consulta para la entidad principal y una adicional por cada elemento relacionado.

C) Que Hibernate no pueda generar SQL.

D) Que la relación pase a ser uno a uno.

18. En una primary key compuesta mapeada con `@EmbeddedId`, la clase embebida que representa la clave suele:

A) No necesitar implementar `equals` ni `hashCode`.

B) Ser una clase simple con los campos clave y métodos `equals` y `hashCode` coherentes.

C) Incluir siempre lógica de negocio compleja.

D) Ser una interfaz sin implementación.

19. En una relación muchos a muchos simple entre `Autor` y `Libro`, ¿qué consulta conceptual te permite obtener todos los libros de un autor desde el código usando Hibernate?

A) Acceder a la colección `autor.getLibros()` si la asociación está mapeada.

B) Ejecutar manualmente un `DELETE` sobre la tabla intermedia.

C) Releer todo el catálogo de libros y filtrarlo a mano.

D) No es posible, Hibernate no soporta relaciones muchos a muchos.

20. ¿Qué opción resume mejor una buena práctica general al trabajar con relaciones y Hibernate en una aplicación Java?

A) Evitar las anotaciones y gestionar todo con SQL manual.

B) Diseñar bien el modelo relacional, mapear correctamente las asociaciones (`@OneToOne`, `@OneToMany`, `@ManyToMany`), y elegir con cuidado la estrategia de carga para evitar consultas innecesarias.

C) Usar siempre carga ansiosa (`EAGER`) en todas las asociaciones.

D) Utilizar una única tabla con todos los datos para no tener relaciones.

Nº	Respuesta	Nº	Respuesta
1	B	11	D
2	C	12	B
3	A	13	A
4	B	14	B
5	C	15	B
6	C	16	C
7	B	17	B
8	B	18	B
9	B	19	A
10	A	20	B

SpringData

1. ¿Cuál es el objetivo principal de Spring Data en una aplicación Spring Boot?

A) Sustituir completamente a la base de datos.

B) Simplificar el acceso a datos proporcionando repositorios y eliminando gran parte del código repetitivo de DAOs.

C) Generar automáticamente interfaces gráficas.

D) Reemplazar al servidor web embebido.

2. ¿Qué interfaz suelen extender los repositorios de Spring Data JPA para disponer de operaciones CRUD básicas?

A) `JdbcTemplate`

B) `JpaEntityManager`

C) `JpaRepository`

D) `RepositoryManager`

3. En una entidad `Cliente`, queremos un repositorio que permita operaciones CRUD y paginación. ¿Qué firma de interfaz es la más típica?

A) `public interface ClienteRepository {}`

B) `public interface ClienteRepository extends JpaRepository<Cliente, Long> {}`

C) `public interface ClienteRepository extends List<Cliente> {}`

D) `public interface ClienteRepository extends Transactional {}`

4. ¿Qué característica clave ofrece Spring Data con los “métodos derivados” (derived query methods)?

A) Generar scripts SQL para crear la base de datos.

B) Cambiar automáticamente de gestor de base de datos sin configuración.

C) Crear consultas a partir del nombre del método, por ejemplo `findByEmail` o `findByNombreAndActivo`.

D) Ejecutar consultas solo en memoria sin usar la base de datos.

5. Dado el método `List<Cliente> findByNombreContaining(String texto);` en un repositorio Spring Data JPA, ¿qué hace este método?

A) Busca clientes cuyo nombre es exactamente igual a `texto`.

B) Busca clientes que no tienen nombre.

C) Busca clientes cuyo nombre contiene la subcadena indicada en `texto`.

D) Elimina clientes por nombre.

6. ¿Qué anotación de Spring se usa habitualmente para definir una consulta personalizada en un repositorio cuando el nombre del método no es suficiente?

A) `@QueryParam`

B) `@Sql`

C) `@Query`

D) `@Native`

7. En Spring Data JPA, ¿qué permite hacer la anotación `@Query` con el atributo `nativeQuery = true`?

A) Ejecutar solo consultas en memoria.

B) Desactivar todas las transacciones.

C) Indicar que la consulta definida es SQL nativo en lugar de JPQL.

D) Obligar a que el método sea de solo lectura.

8. ¿Qué ventaja tiene el uso de interfaces de repositorio frente a escribir manualmente toda la capa DAO con `EntityManager`?

A) El rendimiento siempre es mayor, independientemente del caso.

B) Se reduce drásticamente el código repetitivo y se obtiene de forma automática CRUD, paginación y ordenación.

C) Permite usar Spring sin anotaciones.

D) Elimina la necesidad de configurar la conexión a base de datos.

9. En Spring Data, ¿qué interfaz proporciona métodos básicos como `findAll`, `save`, `deleteById` y se usa como base de otras interfaces?

A) `JpaTemplate`

B) `CrudManager`

C) `CrudRepository`

D) `JdbcRepository`

10. ¿Qué anotación se usa en Spring para indicar que un método (o clase) debe ejecutarse dentro de una transacción?

A) `@Repository`

B) `@Service`

C) `@Transactional`

D) `@PersistenceContext`

11. Según la documentación de Spring Data JPA, ¿cómo se configuran por defecto las transacciones en los métodos heredados de `CrudRepository` / `JpaRepository`?

A) Todos los métodos se ejecutan sin transacción.

B) Los métodos de lectura usan transacción marcada como read-only y los de escritura usan una transacción normal.

C) Solo los métodos con `@Query` son transaccionales.

D) Las transacciones dependen del nombre del método.

12. Si definimos un método de consulta personalizado en un repositorio, por ejemplo `List<Pedido> findByEstado(String estado);`, ¿qué debemos tener en cuenta respecto a las transacciones?

A) Que siempre hereda automáticamente una transacción de escritura.

B) Que nunca se puede ejecutar dentro de una transacción.

C) Que no lleva configuración transaccional por defecto y, si lo necesitamos, debemos anotarlo o encapsularlo en un servicio transaccional.

D) Que obliga a usar SQL nativo.

13. ¿Qué capa suele ser el mejor lugar para colocar los límites de transacción (`@Transactional`) en una aplicación Spring Boot con Spring Data?

A) En cada método del repositorio individualmente.

B) En la capa de servicios, donde se agrupan varias operaciones de repositorio en una unidad de trabajo.

C) Solo en los controladores REST.

D) Únicamente en la clase `SpringBootApplication`.

14. ¿Qué beneficio aporta marcar una transacción como `readOnly = true` en Spring?

A) Impide completamente hacer `SELECT`.

B) Obliga a usar siempre SQL nativo.

C) Permite optimizaciones internas (por ejemplo, menos comprobaciones de cambios y pistas al driver JDBC) en operaciones solo de lectura.

D) Evita que se cierren las conexiones.

15. Dado un repositorio `PedidoRepository extends JpaRepository<Pedido, Long>`, ¿qué hace el método `findByClienteId(Long idCliente);` si se define siguiendo la convención de nombres?

A) Devuelve un único cliente por su id.

B) Elimina todos los pedidos del cliente.

C) Devuelve la lista de pedidos cuyo `cliente.id` coincide con `idCliente`.

D) Actualiza el estado de los pedidos del cliente.

16. ¿Qué anotación se coloca normalmente en la interfaz de repositorio para que Spring la detecte como componente de acceso a datos?

A) `@Component`

B) `@Entity`

C) `@Repository`

D) `@EntityScan`

17. Para paginar resultados con Spring Data JPA, ¿qué tipo de parámetro se añade a la firma del método de repositorio?

A) Un `int` con el número de página.

B) Un `Map<String, Object>` con filtros.

C) Un objeto `Pageable` para indicar página, tamaño y ordenación.

D) Una lista de IDs.

18. ¿Qué tipo de valor devuelven típicamente los métodos de repositorio que quieren ofrecer resultados paginados?

A) `List<T>` obligatoriamente.

B) `Stream<T>`.

C) `Page<T>` o `Slice<T>`, con información adicional de paginación.

D) `Optional<T>`.

19. En una consulta personalizada con `@Query("SELECT p FROM Pedido p WHERE p.fecha >= :inicio AND p.fecha <= :fin")`, ¿qué anotación se usa en los parámetros del método para enlazarlos con `:inicio` y `:fin`?

A) `@ParamValue`

B) `@Param`

C) `@Bind`

D) `@PathVariable`

20. ¿Qué práctica se considera adecuada al combinar Spring Data JPA con transacciones?

A) Anotar todos los métodos de repositorio con `@Transactional` sin analizar la lógica.

B) Evitar siempre las transacciones para mejorar el rendimiento.

C) Definir los límites transaccionales en servicios, usar repositorios para acceso a datos y marcar como `readOnly` las operaciones exclusivamente de lectura.

D) Abrir y cerrar transacciones manualmente con `EntityTransaction` en cada método.

Nº	Respuesta	Nº	Respuesta

1	B	11	B
2	C	12	C
3	B	13	B
4	C	14	C
5	C	15	C
6	C	16	C
7	C	17	C
8	B	18	C
9	C	19	B
10	C	20	C

Las preguntas del base de datos XML están incluidas en el tema 2